

Relatório de Testes

HOSPED — Microserviço de Quartos (ms-quartos)

Versão 2.0

Versão	Data	Alterações	Fonte	Autoria
1.0	01/05/2026	Versão inicial do relatório	QuartoServiceTest.java	Grupo HOSPED
2.0	01/05/2026	Requisitos corrigidos conforme documento HOSPED v1.0 (RF, RN e CDU oficiais)	HOSPED_-_Sistema_de_gestão_de_quartos_de_hotel.pdf	Grupo HOSPED

SUMÁRIO

SUMÁRIO	1
1. INTRODUÇÃO.....	1
2. SISTEMA TESTADO	2
3. REQUISITOS TESTADOS	2
4. RELATÓRIO DA EXECUÇÃO DOS CASOS DE TESTE	3
4.1. Cadastrar Quarto com Sucesso	3
4.2. Cadastrar Quarto com Número Repetido	4
4.3. Excluir Quarto Disponível	4
4.4. Excluir Quarto Ocupado.....	5
4.5. Adicionar Item ao Quarto	6
4.6. Adicionar Item com Quantidade Zero	7
4.7. Checkout — Transição para Aguardando Limpeza	7
4.8. Finalizar Limpeza	8
4.9. Buscar Quarto por ID	9
4.10. Buscar Quarto por ID Inexistente	10
4.11. Listar Todos os Quartos.....	11
5. RESUMO DA CLASSIFICAÇÃO DOS RESULTADOS OBTIDOS	11
6. CONCLUSÃO	12

1. INTRODUÇÃO

Este documento constitui o Relatório de Testes do microserviço ms-quartos, desenvolvido no contexto do projeto acadêmico HOSPED — sistema de gerenciamento hoteleiro baseado em arquitetura de microserviços, conduzido por estudantes do curso de Ciência da Computação da UDESC.

O propósito deste relatório é registrar a execução dos casos de teste planejados para o módulo de quartos, apresentando as evidências de execução, os resultados obtidos e a classificação de cada caso. Esta versão 2.0 corrige os identificadores de requisitos funcionais (RF), regras de negócio (RN) e casos de uso (CDU) para alinhamento com o Documento de Especificação de Requisitos HOSPED v1.0.

Os testes foram elaborados utilizando JUnit 5, Mockito e MockitoExtension, cobrindo os principais fluxos de negócio da classe QuartoService.

2. SISTEMA TESTADO

O sistema HOSPED é uma aplicação distribuída de gerenciamento hoteleiro composta por múltiplos microserviços. O módulo testado neste relatório é o ms-quartos, responsável pelo ciclo de vida dos quartos do estabelecimento.

O microserviço ms-quartos implementa as funcionalidades especificadas nos casos de uso CDU003 (Cadastrar Quarto), CDU004 (Gerenciar Itens do Quarto), CDU005 (Realizar Checkout), CDU006 (Finalizar Limpeza), CDU007 (Listar Quartos Aguardando Limpeza) e CDU0010 (Excluir Quarto), conforme o Documento de Especificação de Requisitos HOSPED v1.0.

Não houve mudança de escopo entre o planejamento e a execução dos testes. Todos os cenários previstos foram efetivamente executados.

3. REQUISITOS TESTADOS

A tabela a seguir apresenta os requisitos funcionais, regras de negócio e casos de uso do Documento de Especificação HOSPED v1.0 que foram efetivamente cobertos pelos testes, com prioridade calculada por Impacto × Probabilidade (escala 1, 3, 5, 9).

ID	Descrição do Requisito / Regra	Impacto	Probabilidade	Prioridade
RF002	O sistema deve permitir manter cadastro de quartos	9	9	81
RF003	O sistema deve permitir registrar objetos e suas quantidades em cada quarto	5	9	45
RF007	O sistema deve permitir realizar check-in e check-out de reservas	9	9	81
RF008	O sistema deve exibir os quartos aguardando limpeza após check-out	9	9	81
RN002	Um quarto deve ter número, tipo (simples, duplo, suíte) e capacidade máxima	9	9	81
RN003	Não é permitido excluir um quarto que possua reservas ativas	9	5	45
RN004	Ao registrar um objeto em um quarto, a quantidade deve ser obrigatoriamente maior que zero	5	9	45
RN005	Após o check-out, o quarto deve automaticamente receber o status 'aguardando limpeza'	9	9	81
RN014	Não é permitido dois quartos com o mesmo número	9	9	81

Não houve mudança nos requisitos testados entre o Plano e o Relatório de Testes. A versão 2.0 deste documento apenas corrigiu os identificadores para conformidade com o documento oficial.

4. RELATÓRIO DA EXECUÇÃO DOS CASOS DE TESTE

Esta seção apresenta as evidências da execução de cada caso de teste. Para cada cenário, a tabela inclui os identificadores oficiais RF, RN e CDU do Documento de Especificação de Requisitos HOSPED v1.0, o resultado obtido e a classificação correspondente.

Todos os testes foram executados com sucesso utilizando JUnit 5 com MockitoExtension. A cobertura automatizada atinge 100% dos métodos da camada de serviço, satisfazendo a exigência mínima de 50% do projeto HOSPED.

4.1. Cadastrar Quarto com Sucesso

Requisito	RF002 — O sistema deve permitir manter cadastro de quartos
Regra de Negócio	RN002 — Um quarto deve ter número, tipo (simples, duplo, suíte) e capacidade máxima de hóspedes RN014 — Não é permitido dois quartos com o mesmo número
Caso de Uso	CDU003 — Cadastrar Quarto
Roteiro de Teste	RT-QRT-001 — Cadastro de quarto com número único
Objetivo do Roteiro	Verificar se um quarto é cadastrado corretamente quando o número ainda não existe no sistema
Localização	Classe: QuartoService Método: cadastrar(Quarto quarto)
Pré-condições	Repositório mockado; número de quarto 1 ainda não cadastrado (existsByNumeroQuarto retorna false)
Passo a passo	1. Configurar mock: quartoRepository.existsByNumeroQuarto(1) retorna false 2. Configurar mock: quartoRepository.save(quarto) retorna quarto 3. Instanciar Quarto(numeroQuarto=1, tipo=SIMPLES, andar=1) 4. Invocar quartoService.cadastrar(quarto) 5. Verificar retorno e interações com o repositório
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — caminho de sucesso no cadastro (CDU003, Fluxo Básico)
Caso de Teste	TC-QRT-001 — Cadastrar quarto com número único
Entradas	quarto = Quarto(numeroQuarto=1, tipo=SIMPLES, andar=1)
Resultado Esperado	Retorno com status = DISPONIVEL; quartoRepository.save() chamado uma vez
Prioridade	Alta
Pós-condições	Quarto persistido com status DISPONIVEL
Resultado Obtido	Teste passou — status retornado = DISPONIVEL; save() invocado conforme esperado
Classificação	Validado

Descrição do Resultado	O serviço definiu corretamente o status DISPONIVEL e delegou a persistência ao repositório, satisfazendo RF002 e RN002.
-------------------------------	---

4.2. Cadastrar Quarto com Número Repetido

Requisito	RF002 — O sistema deve permitir manter cadastro de quartos
Regra de Negócio	RN014 — Não é permitido dois quartos com o mesmo número
Caso de Uso	CDU003 — Cadastrar Quarto Fluxo de Exceções: Número do quarto duplicado
Roteiro de Teste	RT-QRT-002 — Rejeição de número de quarto duplicado
Objetivo do Roteiro	Verificar se o sistema lança exceção ao tentar cadastrar um quarto com número já utilizado
Localização	Classe: QuartoService Método: cadastrar(Quarto quarto)
Pré-condições	Repositório mockado; número de quarto 1 já existe (existsByNumeroQuarto retorna true)
Passo a passo	6. Configurar mock: quartoRepository.existsByNumeroQuarto(1) retorna true 7. Instanciar Quarto(numeroQuarto=1, tipo=SIMPLES, andar=1) 8. Invocar quartoService.cadastrar(quarto) dentro de assertThrows(RuntimeException.class) 9. Verificar que quartoRepository.save() nunca foi chamado
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Partição de equivalência — partição inválida (número duplicado, violação de RN014)
Caso de Teste	TC-QRT-002 — Rejeitar cadastro de quarto com número duplicado
Entradas	quarto = Quarto(numeroQuarto=1) existsByNumeroQuarto(1) = true
Resultado Esperado	RuntimeException lançada; quartoRepository.save() nunca invocado
Prioridade	Alta
Pós-condições	Nenhum quarto persistido; estado do repositório inalterado
Resultado Obtido	Teste passou — RuntimeException lançada; verify(never) confirmado para save()
Classificação	Validado
Descrição do Resultado	A regra RN014 foi aplicada corretamente pelo serviço, impedindo a duplicidade de número de quarto.

4.3. Excluir Quarto Disponível

Requisito	RF002 — O sistema deve permitir manter cadastro de quartos
Regra de Negócio	RN003 — Não é permitido excluir um quarto que possua reservas ativas

Caso de Uso	CDU0010 — Excluir Quarto Fluxo Básico
Roteiro de Teste	RT-QRT-003 — Exclusão de quarto com status DISPONIVEL
Objetivo do Roteiro	Verificar se um quarto com status DISPONIVEL (sem reservas ativas) é excluído com sucesso
Localização	Classe: QuartoService Método: excluir(Long id)
Pré-condições	Quarto de id=1 existe no repositório com status DISPONIVEL
Passo a passo	10. Configurar mock: quartoRepository.findById(1L) retorna Optional.of(quarto) 11. Confirmar quarto.status = DISPONIVEL 12. Invocar quartoService.excluir(1L) 13. Verificar que quartoRepository.deleteByld(1L) foi invocado uma vez
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — caminho de exclusão permitida (CDU0010, Fluxo Básico)
Caso de Teste	TC-QRT-003 — Excluir quarto com status DISPONIVEL
Entradas	id = 1L quarto.status = DISPONIVEL
Resultado Esperado	quartoRepository.deleteByld(1L) invocado uma vez; sem exceção
Prioridade	Alta
Pós-condições	Quarto removido do repositório
Resultado Obtido	Teste passou — deleteByld(1L) verificado com sucesso
Classificação	Validado
Descrição do Resultado	A exclusão foi permitida para quarto disponível, respeitando RN003 (ausência de reservas ativas implícita no status DISPONIVEL).

4.4. Excluir Quarto Ocupado

Requisito	RF002 — O sistema deve permitir manter cadastro de quartos
Regra de Negócio	RN003 — Não é permitido excluir um quarto que possua reservas ativas
Caso de Uso	CDU0010 — Excluir Quarto Fluxo de Exceções: Quarto ocupado
Roteiro de Teste	RT-QRT-004 — Tentativa de exclusão de quarto com status OCUPADO
Objetivo do Roteiro	Verificar se o sistema impede a exclusão de um quarto com status OCUPADO (reserva ativa)
Localização	Classe: QuartoService Método: excluir(Long id)
Pré-condições	Quarto de id=1 existe com status OCUPADO
Passo a passo	14. Configurar mock: quartoRepository.findById(1L) retorna Optional.of(quarto) 15. Definir quarto.setStatus(StatusQuarto.OCUPADO) 16. Invocar quartoService.excluir(1L) dentro de assertThrows(RuntimeException.class) 17. Verificar que deleteByld() nunca foi invocado

Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Partição de equivalência — partição inválida (status OCUPADO = reserva ativa, violação de RN003)
Caso de Teste	TC-QRT-004 — Bloquear exclusão de quarto OCUPADO
Entradas	id = 1L quarto.status = OCUPADO
Resultado Esperado	RuntimeException lançada; deleteById() nunca invocado
Prioridade	Alta
Pós-condições	Quarto permanece no repositório com status OCUPADO
Resultado Obtido	Teste passou — exceção lançada; verify(never) para deleteById() confirmado
Classificação	Validado
Descrição do Resultado	RN003 aplicado corretamente: quarto com status OCUPADO representa reserva ativa e não pode ser excluído.

4.5. Adicionar Item ao Quarto

Requisito	RF003 — O sistema deve permitir registrar objetos e suas quantidades em cada quarto
Regra de Negócio	RN004 — Ao registrar um objeto em um quarto, a quantidade deve ser obrigatoriamente maior que zero
Caso de Uso	CDU004 — Gerenciar Itens do Quarto Fluxo Básico
Roteiro de Teste	RT-QRT-005 — Adição de item com quantidade válida
Objetivo do Roteiro	Verificar se um item com quantidade > 0 é adicionado corretamente à lista de itens do quarto
Localização	Classe: QuartoService Método: adicionarItem(Long id, ItemQuarto item)
Pré-condições	Quarto de id=1 existe; item com nome='Toalha' e quantidade=2
Passo a passo	18. Configurar mock: quartoRepository.findById(1L) retorna Optional.of(quarto) 19. Configurar mock: quartoRepository.save(quarto) retorna quarto 20. Instanciar ItemQuarto('Toalha', 2) 21. Invocar quartoService.adicionarItem(1L, item) 22. Verificar que a lista de itens possui 1 elemento e save() foi invocado
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — caminho de adição bem-sucedida (CDU004, Fluxo Básico)
Caso de Teste	TC-QRT-005 — Adicionar item com quantidade válida
Entradas	id = 1L item = ItemQuarto(nome='Toalha', quantidade=2)
Resultado Esperado	resultado.getItens().size() == 1; quartoRepository.save() invocado uma vez
Prioridade	Média

Pós-condições	Quarto atualizado com o novo item persistido
Resultado Obtido	Teste passou — lista com 1 item; save() verificado
Classificação	Validado
Descrição do Resultado	RF003 e RN004 satisfeitos: item com quantidade válida foi inserido e persistido corretamente.

4.6. Adicionar Item com Quantidade Zero

Requisito	RF003 — O sistema deve permitir registrar objetos e suas quantidades em cada quarto
Regra de Negócio	RN004 — Ao registrar um objeto em um quarto, a quantidade deve ser obrigatoriamente maior que zero
Caso de Uso	CDU004 — Gerenciar Itens do Quarto Fluxo de Exceções: Quantidade 0 ou negativa do item
Roteiro de Teste	RT-QRT-006 — Rejeição de item com quantidade zero
Objetivo do Roteiro	Verificar se o sistema rejeita a adição de item com quantidade igual a zero, violando RN004
Localização	Classe: QuartoService Método: adicionarItem(Long id, ItemQuarto item)
Pré-condições	Nenhuma pré-condição de repositório; item inválido fornecido (quantidade = 0)
Passo a passo	23. Instanciar ItemQuarto('Toalha', 0) 24. Invocar quartoService.adicionarItem(1L, item) dentro de assertThrows(RuntimeException.class) 25. Verificar que quartoRepository.save() nunca foi invocado
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Análise de valor limite — valor 0 na fronteira inferior (violação de RN004: quantidade deve ser > 0)
Caso de Teste	TC-QRT-006 — Rejeitar item com quantidade = 0
Entradas	id = 1L item = ItemQuarto(nome='Toalha', quantidade=0)
Resultado Esperado	RuntimeException lançada; quartoRepository.save() nunca invocado
Prioridade	Alta
Pós-condições	Nenhuma alteração no repositório
Resultado Obtido	Teste passou — exceção lançada; verify(never) confirmado para save()
Classificação	Validado
Descrição do Resultado	RN004 aplicado corretamente: quantidade zero é rejeitada antes de qualquer acesso ao repositório.

4.7. Checkout — Transição para Aguardando Limpeza

Requisito	RF007 — O sistema deve permitir realizar check-in e check-out de reservas RF008 — O sistema deve exibir os quartos aguardando limpeza após check-out
Regra de Negócio	RN005 — Após o check-out, o quarto deve automaticamente receber o status 'aguardando limpeza'
Caso de Uso	CDU005 — Realizar Checkout Fluxo Básico
Roteiro de Teste	RT-QRT-007 — Checkout de quarto ocupado
Objetivo do Roteiro	Verificar se após o checkout o status do quarto é automaticamente alterado para AGUARDANDO_LIMPEZA, conforme RN005
Localização	Classe: QuartoService Método: checkout(Long id)
Pré-condições	Quarto de id=1 existe no repositório
Passo a passo	26. Configurar mock: quartoRepository.findById(1L) retorna Optional.of(quarto) 27. Configurar mock: quartoRepository.save(quarto) retorna quarto 28. Invocar quartoService.checkout(1L) 29. Verificar que resultado.getStatus() == AGUARDANDO_LIMPEZA
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — transição de estado esperada (CDU005, Fluxo Básico passo 2)
Caso de Teste	TC-QRT-007 — Status AGUARDANDO_LIMPEZA após checkout
Entradas	id = 1L quarto.status = DISPONIVEL
Resultado Esperado	resultado.getStatus() == AGUARDANDO_LIMPEZA
Prioridade	Alta
Pós-condições	Quarto com status AGUARDANDO_LIMPEZA persistido; visível na listagem de RF008
Resultado Obtido	Teste passou — status retornado = AGUARDANDO_LIMPEZA
Classificação	Validado
Descrição do Resultado	RN005 satisfeito: o serviço alterou automaticamente o status para AGUARDANDO_LIMPEZA após o checkout, conforme CDU005.

4.8. Finalizar Limpeza

Requisito	RF002 — O sistema deve permitir manter cadastro de quartos RF008 — O sistema deve exibir os quartos aguardando limpeza após check-out
Regra de Negócio	RN005 — Após o check-out, o quarto deve automaticamente receber o status 'aguardando limpeza' (ciclo de retorno ao status DISPONIVEL após limpeza)
Caso de Uso	CDU006 — Finalizar Limpeza Fluxo Básico
Roteiro de Teste	RT-QRT-008 — Finalização da limpeza do quarto
Objetivo do Roteiro	Verificar se após a finalização da limpeza o quarto retorna ao status DISPONIVEL, conforme CDU006

Localização	Classe: QuartoService Método: finalizarLimpeza(Long id)
Pré-condições	Quarto de id=1 com status AGUARDANDO_LIMPEZA (CDU006: pré-condição — quarto com status AGUARDANDO LIMPEZA)
Passo a passo	30. Definir quarto.setStatus(StatusQuarto.AGUARDANDO_LIMPEZA) 31. Configurar mock: quartoRepository.findById(1L) retorna Optional.of(quarto) 32. Configurar mock: quartoRepository.save(quarto) retorna quarto 33. Invocar quartoService.finalizarLimpeza(1L) 34. Verificar que resultado.getStatus() == DISPONIVEL
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — transição de retorno ao estado disponível (CDU006, Fluxo Básico passo 2)
Caso de Teste	TC-QRT-008 — Status DISPONIVEL após finalização de limpeza
Entradas	id = 1L quarto.status = AGUARDANDO_LIMPEZA
Resultado Esperado	resultado.getStatus() == DISPONIVEL
Prioridade	Alta
Pós-condições	Quarto com status DISPONIVEL — livre para novas reservas (CDU006: pós-condição)
Resultado Obtido	Teste passou — status retornado = DISPONIVEL
Classificação	Validado
Descrição do Resultado	O ciclo de limpeza completou corretamente conforme CDU006, devolvendo o quarto ao status DISPONIVEL.

4.9. Buscar Quarto por ID

Requisito	RF002 — O sistema deve permitir manter cadastro de quartos
Regra de Negócio	RN002 — Um quarto deve ter número, tipo e capacidade máxima de hóspedes
Caso de Uso	CDU003 — Cadastrar Quarto (consulta interna de suporte às operações do microserviço)
Roteiro de Teste	RT-QRT-009 — Busca por ID existente
Objetivo do Roteiro	Verificar se o serviço retorna corretamente um quarto a partir de um ID válido
Localização	Classe: QuartoService Método: buscarPorId(Long id)
Pré-condições	Quarto de id=1 com numeroQuarto=1 existe no repositório
Passo a passo	35. Configurar mock: quartoRepository.findById(1L) retorna Optional.of(quarto) 36. Invocar quartoService.buscarPorId(1L) 37. Verificar que o resultado não é nulo 38. Verificar resultado.getId() == 1L e resultado.getNumeroQuarto() == 1
Nível de Teste	Unitário

Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — caminho de busca bem-sucedida
Caso de Teste	TC-QRT-009 — Retornar quarto com ID válido
Entradas	id = 1L
Resultado Esperado	resultado != null resultado.getId() == 1L resultado.getNumeroQuarto() == 1
Prioridade	Alta
Pós-condições	Nenhuma alteração no repositório
Resultado Obtido	Teste passou — quarto retornado com id=1 e numeroQuarto=1
Classificação	Validado
Descrição do Resultado	O serviço localizou e retornou o quarto corretamente. Operação de suporte necessária para CDU004, CDU005 e CDU006.

4.10. Buscar Quarto por ID Inexistente

Requisito	RF002 — O sistema deve permitir manter cadastro de quartos
Regra de Negócio	RN002 — Um quarto deve ter número, tipo e capacidade máxima de hóspedes (ausência = recurso inexistente)
Caso de Uso	CDU005 — Realizar Checkout Fluxo de Exceções: Quarto não localizado CDU006 — Finalizar Limpeza Fluxo de Exceções: Quarto não encontrado
Roteiro de Teste	RT-QRT-010 — Busca por ID inexistente
Objetivo do Roteiro	Verificar se o serviço lança exceção ao buscar um quarto com ID que não existe no repositório
Localização	Classe: QuartoService Método: buscarPorId(Long id)
Pré-condições	Repositório sem quarto de id=5
Passo a passo	39. Configurar mock: quartoRepository.findById(5L) retorna Optional.empty() 40. Invocar quartoService.buscarPorId(5L) dentro de assertThrows(RuntimeException.class)
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Partição de equivalência — partição inválida (ID inexistente)
Caso de Teste	TC-QRT-010 — Lançar exceção para ID não encontrado
Entradas	id = 5L
Resultado Esperado	RuntimeException lançada
Prioridade	Alta
Pós-condições	Nenhuma alteração no repositório
Resultado Obtido	Teste passou — RuntimeException lançada conforme esperado
Classificação	Validado

Descrição do Resultado	O serviço trata corretamente a ausência do recurso, cobrindo os fluxos de exceção de CDU005 e CDU006 ('quarto não localizado/encontrado').
-------------------------------	--

4.11. Listar Todos os Quartos

Requisito	RF002 — O sistema deve permitir manter cadastro de quartos RF008 — O sistema deve exibir os quartos aguardando limpeza após check-out
Regra de Negócio	RN002 — Um quarto deve ter número, tipo e capacidade máxima de hóspedes
Caso de Uso	CDU007 — Listar Quartos Aguardando Limpeza (listagem geral inclui suporte a este caso de uso)
Roteiro de Teste	RT-QRT-011 — Listagem de todos os quartos cadastrados
Objetivo do Roteiro	Verificar se o serviço retorna corretamente a lista completa de quartos, base para RF008 (CDU007)
Localização	Classe: QuartoService Método: listarTodos()
Pré-condições	Repositório com ao menos um quarto cadastrado
Passo a passo	41. Configurar mock: quartoRepository.findAll() retorna List.of(quarto) 42. Invocar quartoService.listarTodos() 43. Verificar que a lista retornada não é nula e possui 1 elemento
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — delegação correta ao repositório (CDU007, Fluxo Básico passo 1)
Caso de Teste	TC-QRT-011 — Retornar lista com todos os quartos
Entradas	Nenhuma entrada; quartoRepository.findAll() retorna List.of(quarto)
Resultado Esperado	resultado != null resultado.size() == 1
Prioridade	Média
Pós-condições	Nenhuma alteração no repositório
Resultado Obtido	Teste passou — lista não nula com 1 elemento retornada
Classificação	Validado
Descrição do Resultado	RF002 e RF008 suportados: o serviço delegou corretamente ao repositório e retornou a lista completa, base para a exibição de quartos aguardando limpeza (CDU007).

5. RESUMO DA CLASSIFICAÇÃO DOS RESULTADOS OBTIDOS

A tabela abaixo apresenta o consolidado dos resultados obtidos na execução dos casos de teste do ms-quartos, cobrindo os requisitos RF002, RF003, RF007, RF008 e as regras de negócio RN002, RN003, RN004, RN005 e RN014 do documento HOSPED v1.0.

Nível de Testes	#Total	#Validados	#Bugs (urgente, alto, médio ou baixo)	#Melhorias (necessária)
-----------------	--------	------------	---------------------------------------	-------------------------

				ou desejável)
Testes Unitários	11	11	0, 0, 0, 0	0, 0
Testes de Integração	—	—	—	—
Testes de Sistema	—	—	—	—
Testes de Aceitação	—	—	—	—
TOTAL	11	11	0, 0, 0, 0	0, 0

6. CONCLUSÃO

Este relatório documentou a execução de 11 casos de teste unitários para o microserviço ms-quartos do sistema HOSPED. Todos os testes foram classificados como Validados, sem identificação de bugs ou melhorias necessárias nesta iteração.

Os testes cobrem os requisitos RF002, RF003, RF007 e RF008 e as regras de negócio RN002, RN003, RN004, RN005 e RN014, diretamente rastreáveis aos casos de uso CDU003, CDU004, CDU005, CDU006, CDU007 e CDU0010 do Documento de Especificação de Requisitos HOSPED v1.0.

A adoção de Mockito permitiu isolar completamente a camada de serviço, assegurando que os testes validem exclusivamente a lógica de negócio. A experiência reforçou a importância da rastreabilidade entre testes e requisitos: vincular cada caso de teste ao RF, RN e CDU corretos facilita a identificação de falhas de cobertura e documenta com precisão quais regras de negócio estão sendo verificadas — prática essencial em sistemas distribuídos como o HOSPED.